Music Recommendation System

Team 2 – Manas Jashnani and Mustafa Kashif

1. Introduction

Music recommendation systems are crucial for enhancing user experience on streaming platforms. This project explores two independent methodologies for music recommendation:

a) Lyrics-Based Recommendation: Uses song lyrics to recommend similar tracks by analysing textual themes.
b) Content-Based Recommendation: Employs MP3 audio content to generate mel-spectrograms for feature extraction using convolutional neural networks (CNNs).

While the lyrics-based system encountered challenges due to undertrained models, the content-based system achieved promising results and serves as the primary focus of this report.

2. Background and Problem Statement

With the vast number of songs available online, providing personalized and relevant music recommendations is a complex problem. Traditional collaborative filtering models depend heavily on user interactions and suffer from the cold-start problem. Content-aware systems, on the other hand, rely on the intrinsic properties of the items (songs) themselves.

- Lyrics-Based Challenges: Capturing meaningful similarities from textual data.
- Content-Based Challenges: Extracting relevant features from audio signals.

This project attempts two parallel paths, one using text (lyrics) and another using audio (content).

3. Related Work

- **Pandora's Music Genome Project**:
  Utilizes expert-curated audio features and metadata to create a "genome" of musical attributes, forming the basis for fine-grained, content-based recommendations that capture subtle stylistic elements.
- **Spotify's Content+Collaboration Hybrid**:
  Combines audio embeddings extracted from deep learning models with extensive user-item interaction data, blending content analysis and collaborative filtering to offer highly personalized playlists and song suggestions.
- **YouTube Music's Audio-Visual Integration**:
  Leverages audio fingerprints, user engagement metrics, and even video-related metadata to suggest music that aligns with listener preferences, bridging content-based features with consumption patterns on a global scale.
- **Apple Music's Curator-Enhanced Recommendations**:
  Employs proprietary audio analysis, editorial playlists, and expert curation, merging algorithmic insights with human-driven content to produce tailored recommendations across diverse genres and listener demographics.
- **SoundCloud's Community-Driven Discovery**:
  Incorporates waveform-level audio analysis, user-uploaded tags, and engagement feedback, enabling emerging artists and niche genres to surface through content-based matches augmented by social listening behaviors.

4. Methodology

4.1. Lyrics-Based System

- Data Preprocessing: Tokenization and vectorization of lyrics.
- LDA Topic Modeling: Extracts semantic structures, outputting vectors representing song themes.
- MPNet embeddings: Embeddings generated for each lyrics to capture semantics of the song lyrics.
- Dimensionality Reduction: Autoencoder reduces vectors for clustering.
- Clustering: HDBSCAN groups songs based on proximity in the reduced space.

4.2. Content-Based System

4.2.1 Data Preparation and Splitting

We used the **fma_small** dataset, which provides 30s mp3 tracks from various genres. To ensure robust model training and fair evaluation, we split the dataset as follows:

- **80% Training/Testing/Validation Set**: This subset was used to train and evaluate the CNN model. By grouping and stratifying songs by genre, we ensured that each genre was well-represented. This balanced approach helped the model learn genre distinctions more effectively.
- **20% Recommendation Set**: The remaining subset of the dataset was set aside specifically for the recommendation engine. After the model had been trained and validated, these tracks were used exclusively to generate recommendations for end users, ensuring that the model's evaluation and the recommendation pool remained distinct.

4.2.2 Preprocessing and Renaming

To handle the separate recommendation set, we created a dedicated folder. We then **renamed the MP3 files** based on metadata (title, artist, and album) using a script that read ID3 tags. This ensured consistent, descriptive filenames, improving dataset organization and simplifying subsequent retrieval and display on the web interface.

4.2.3 Spectrogram Creation

Raw audio data is not directly suitable for CNN input. Thus, we employed the **Librosa** library to transform each audio track into a **mel-spectrogram**, which is a visual time-frequency representation of the sound. These spectrogram images capture harmonics and rhythmic patterns essential for genre and style discrimination. Each spectrogram was stored as a grayscale image file.

4.2.4 Spectrogram Slicing

To standardize input sizes for the CNN, we **sliced the spectrograms into fixed 128x128 patches**. This ensured uniform input dimensions and effectively augmented the dataset by producing multiple training examples per track. The slicing approach allowed the model to focus on local patterns and reduced the risk of overfitting to specific segments.

4.2.5 CNN Model Architecture and Training

We designed and implemented a CNN architecture with the following characteristics:

- **Layers**: Multiple convolutional layers with ReLU activation and batch normalization, followed by max pooling layers to progressively reduce the spatial dimensions.

- **Fully Connected Layers**: After flattening the extracted features, dense layers combined with dropout improved generalization.
- **Data Loading and Normalization**: The images were normalized and split into training and validation sets. We used callbacks such as early stopping and learning rate reduction on plateau to ensure efficient and stable training.
- **Evaluation**: The model's performance was monitored on a hold-out test set, ensuring that the trained network generalized well to unseen data.

4.2.6 Feature Extraction Using the Second-to-Last Layer

Instead of using the model's final classification output, we extracted the **latent feature vectors from the second-to-last fully connected layer** of the trained CNN. These feature vectors represented high-level audio attributes learned during training and served as compact embeddings for each song.

4.2.7 Precomputing and Storing Feature Vectors

For each track, we applied the CNN to every spectrogram slice and averaged the resulting embeddings to produce a single representative vector. We then stored these vectors on disk. By caching this data, we eliminated the need for repetitive model inference at recommendation time, enabling swift real-time suggestions.

4.2.8 Computing Similarities for Recommendations

Using the precomputed feature vectors, we employed **cosine similarity** to measure how closely related two songs are in the learned feature space. Sorting these values allowed us to easily retrieve the top N most similar songs (commonly top 2) for the user's selected "seed" song.

- **Surprise Me Feature**: The system also supports a "Surprise Me" option. Initially, it selects the track with the least similarity to the seed song, providing a starkly different recommendation. For subsequent surprises, it filters the candidate set to songs with mid-range similarity (0 to 0.6), offering novel yet somewhat related options to users.

5. Implementation

Lyrics-Based:

Frontend

Recommendations displayed via a basic web interface built using HTML and CSS.

Backend

Integrated pipelines for LDA, MPNet embeddings, autoencoders, and clustering.

Flask / FastAPI backend to expose the recommendation systems via APIs

Deployment

Development environment: Local Python setup.

Content-Based:

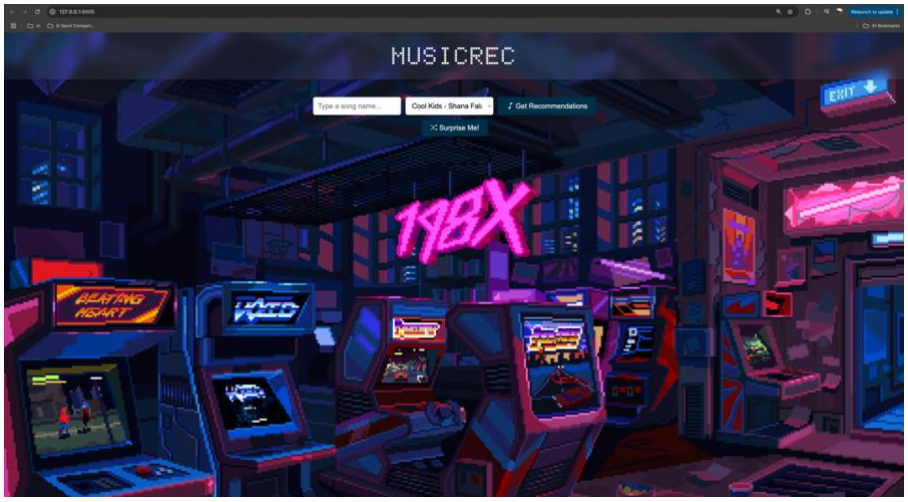
Backend Scripts

We structured our code into separate modules:

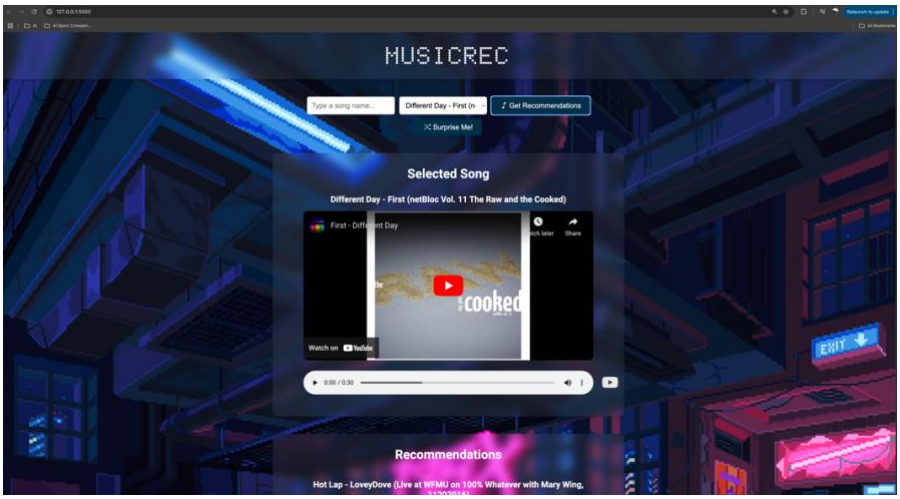
- **Data Splitting & Renaming (datasplit.py, rename.py):** Organized files, ensured balanced genre distributions, and renamed MP3s.
- **Spectrogram Generation & Slicing (data_importer.py, spectogram_slicer.py):** Created mel-spectrograms and sliced them into uniform patches.
- **Data Loading and Model Training (data_loads.py, train.py):** Loaded preprocessed images, trained the CNN, and saved the model and training history.
- **Precomputation of Features (precompute_features.py):** Extracted latent features from the trained model and stored them.
- **Recommendation System (recommendation.py):** Computed cosine similarity scores and generated the final recommendations.

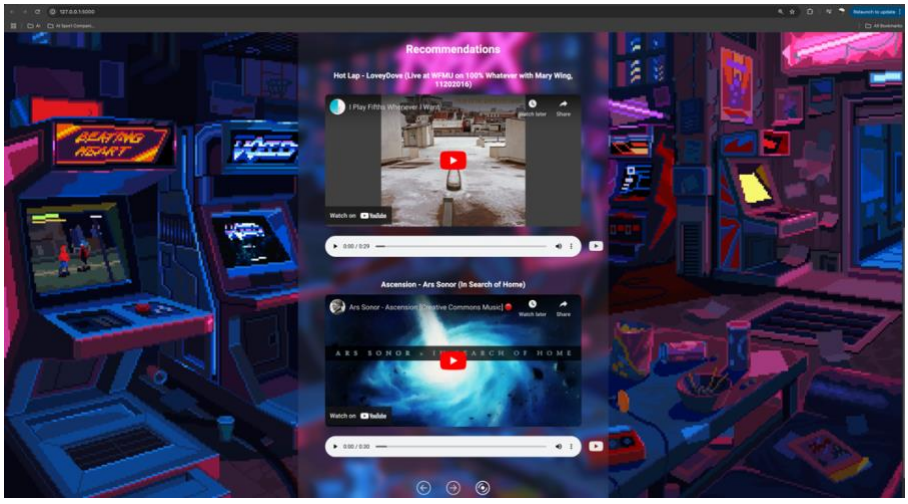
Web Interface and User Interaction

We developed a lightweight **Flask-based web application** to allow users to interact seamlessly with the recommendation system:



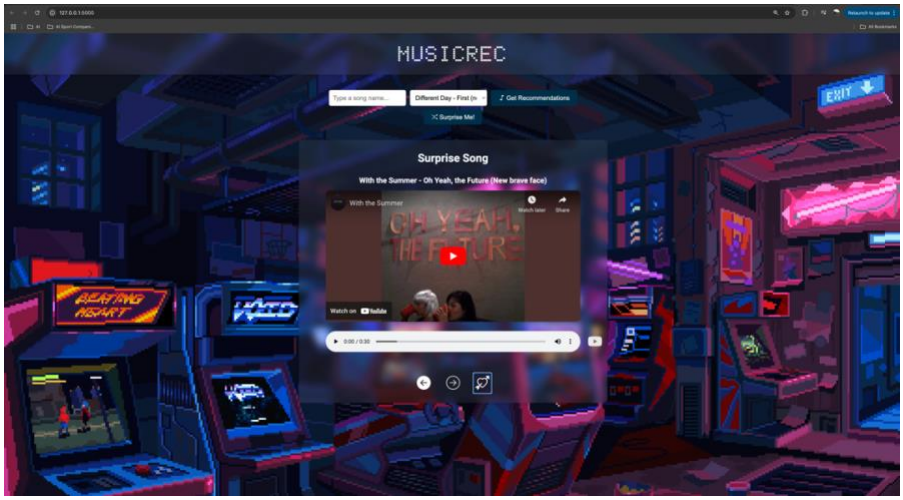
- **Search and Filter**: Users can type queries into a search bar to filter and locate a seed track from a dropdown menu.
- **Recommendations**:





After the user selects a seed song, the system displays it alongside an embedded YouTube video (if available) and a playable audio preview. It then generates the top 2 similar tracks using the precomputed feature embeddings. The user can navigate through these recommendations and request 2 new ones with a “One More Time” button(Record Icon). Recommendations are the songs with the highest cosine similarity score.

- **Surprise Me Feature**:



To enhance discovery beyond merely similar tracks, a “Surprise Me” option is provided. Initially, this feature presents the least similar track from the recommendation set, offering a stark contrast to the seed song. For subsequent surprise requests, the system filters potential candidates for those with moderate similarity (0 to 0.6), balancing novelty with some degree of familiarity. This approach ensures that users can break away from strictly similar recommendations and explore more diverse musical landscapes.

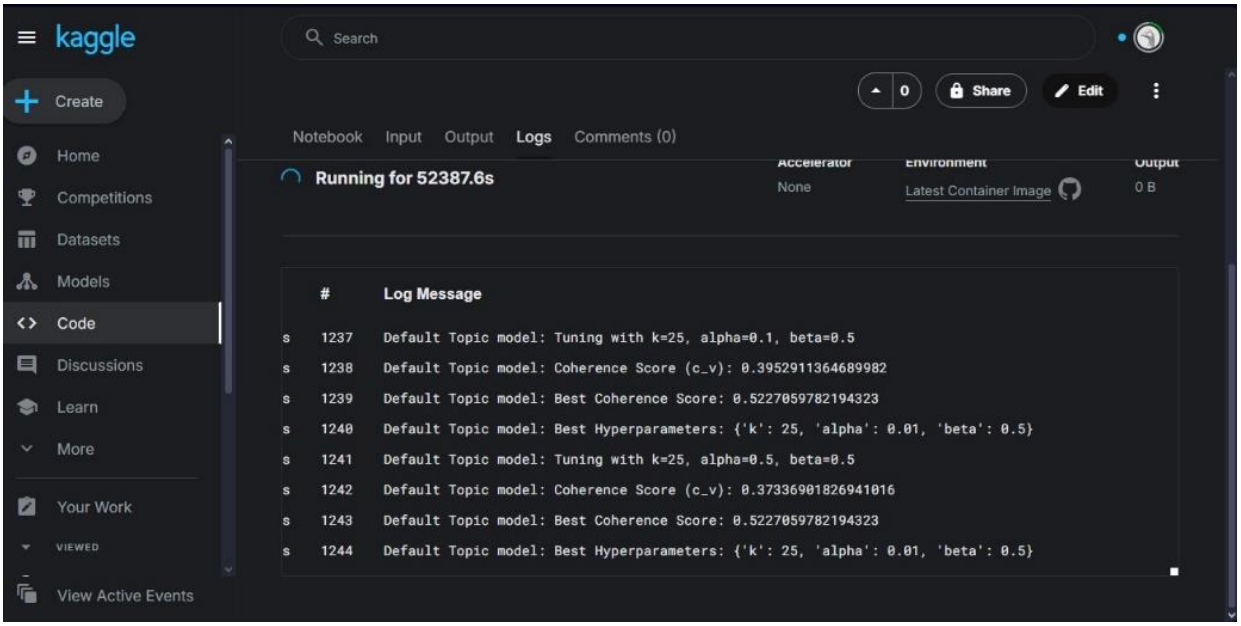
- **YouTube Integration**: Each recommended song entry includes a link to the corresponding YouTube video, allowing users to discover more about the track and artist.
- **User Navigation**: The interface provides buttons for navigating forward and backward through previously viewed recommendations and generating additional suggestions on demand. The precomputed embeddings ensure responsiveness and fluid interaction.

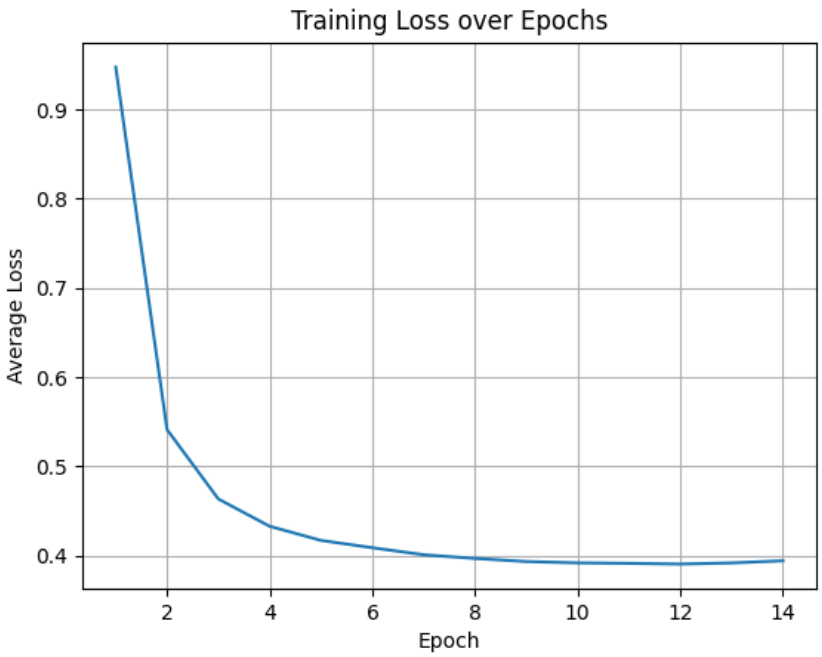
6. Testing and Evaluation

Lyrics-Based

Results were inconsistent due to an undertrained autoencoder, causing random recommendations.

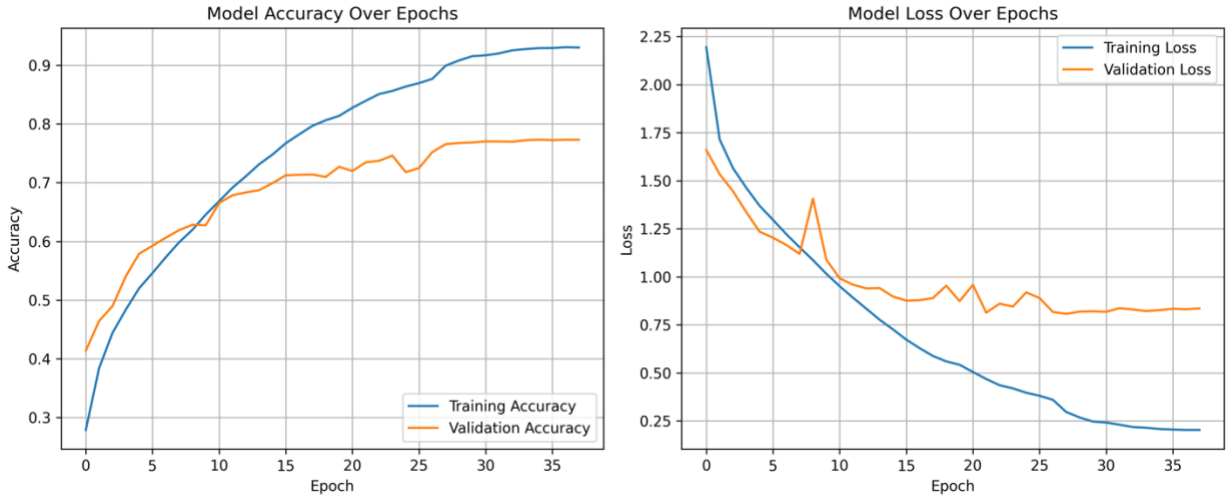
Training Loss: 0.37 at epoch 10, which requires further tuning.





Content-Based

1. CNN Training History and Performance Metrics



The training history shows that both training and validation accuracies increased steadily over epochs, as depicted in the **Accuracy and Loss Graphs**. Initially, the training accuracy lagged behind the validation accuracy, indicating that the model was generalizing reasonably well even in early stages. Over time, the training accuracy surpassed the validation accuracy, approaching ~92-93%, while the validation accuracy stabilized around ~77%. Simultaneously, training and validation losses decreased, though validation loss plateaued, reflecting diminishing returns on further training and hinting at a near-converged state.

2. Classification Report and Weighted F1 Score

	precision	recall	f1-score	support
Hip-Hop	0.78	0.86	0.82	535
International	0.80	0.77	0.79	584
Electronic	0.81	0.78	0.80	620
Folk	0.79	0.79	0.79	634
Experimental	0.76	0.75	0.76	557
Rock	0.79	0.81	0.80	583
Pop	0.64	0.59	0.61	666
Instrumental	0.76	0.79	0.78	625
accuracy			0.77	4804
macro avg	0.77	0.77	0.77	4804
weighted avg	0.76	0.77	0.76	4804

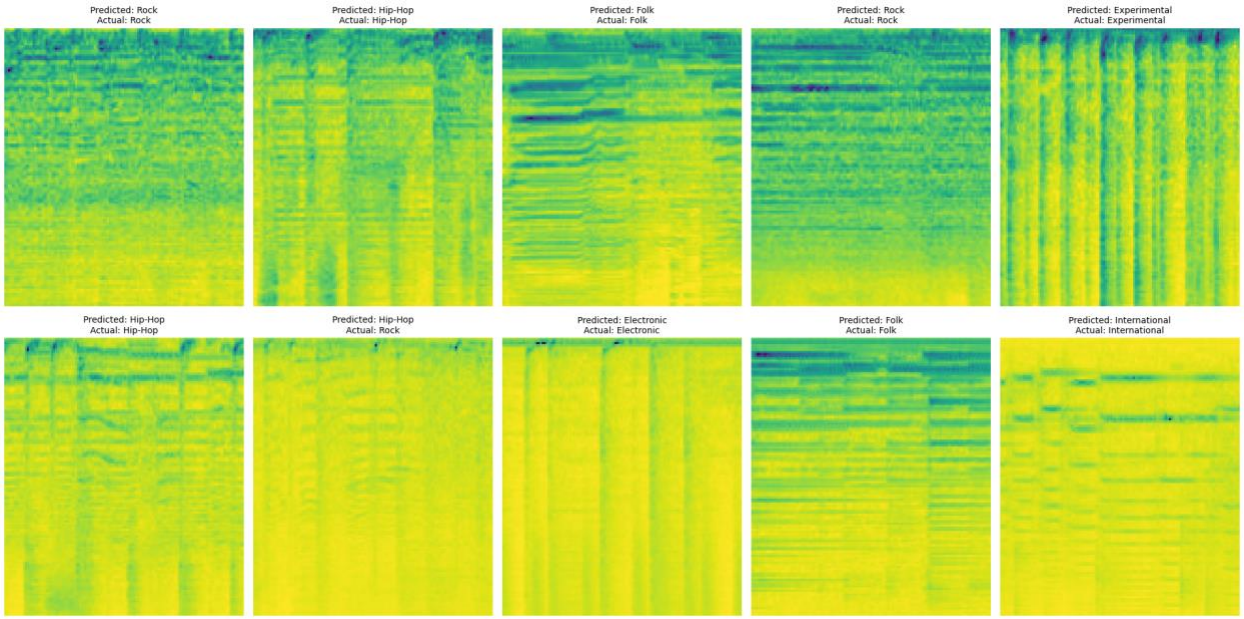
The final **Classification Report** and a **Weighted F1 Score of 0.7649** underscore the model's reasonably strong performance in classifying multiple genres. Precision and recall values hover around 0.75–0.80 for most genres, and the accuracy reaches ~0.77 on a test set of 4,804 samples. Certain genres like Pop (f1-score ~0.61) proved more challenging, possibly due to overlapping characteristics with other categories. Nonetheless, the weighted F1 score close to 0.77 signifies that, on average, the model made consistent and well-balanced predictions across the genre spectrum.

3. Confusion Matrix Analysis



The **Confusion Matrix** provides insight into genre-level misclassifications. While genres such as "Folk," "Instrumental," "Electronic," and "Rock" generally achieved robust predictions, some confusion arose between closely related genres. For instance, "Pop" tracks were sometimes misclassified into neighboring genres, suggesting shared audio features or stylistic overlaps.

4. Spectrogram-Based Predictions



Visualizations of **Spectrogram Predictions** reveal how the model operates in practice. Each spectrogram patch corresponds to a time-frequency slice of a song's audio. Correctly classified examples show a strong alignment between actual and predicted genres, while misclassifications highlight the inherent difficulty of identifying subtle cues.

These spectrogram images also confirm that the model relies on visually discernible patterns in the time-frequency domain. While not directly interpretable in human musical terms, they provide a tangible way to understand how CNNs "see" and classify music.

7. Limitations/Threats and Future Work

1. **Lyrics-Based:**
   - Poor performance due to limited training of the autoencoder.
   - Future improvement through larger datasets and hyperparameter tuning.

2. **Content-Based:**

   - **Data Limitations**: Recommendations are strictly limited to the provided dataset. The system cannot introduce new tracks outside the known pool, hindering diversity.

   - **Genre and Language Coverage**: The model learns only from the genres and languages present in the training data. Unseen styles or languages cannot be adequately recommended.

   - **Scalability**: Computing similarities for very large catalogs (millions of songs) can be computationally expensive. Practical, large-scale systems often require hybrid approaches and indexing techniques.

   - **Need for a Hybrid System**: A pure content-based approach does not incorporate user behavior or external metadata. Combining content-based methods with collaborative filtering, or leveraging more advanced techniques, could provide richer, more personalized recommendations while remaining efficient and scalable.

8. Conclusion

This project demonstrates the potential of a content-based recommendation system using CNN-derived embeddings of spectrograms. While the lyrics-based system struggled to produce coherent recommendations, the content-based approach effectively captured audio characteristics and provided meaningful suggestions. Although currently limited to a modest dataset and lacking hybridization with other recommendation paradigms, this system serves as a solid foundation. Future enhancements might integrate user interaction data or collaborate with external databases, ultimately leading to a more comprehensive and scalable hybrid solution.

9. References

- Song Recommendation System Using Lyrics and Audio.
- Emotion-Based Music Recommendation System.
- Overview of Music-Lyrics Analysis Projects.
- Enhanced Music Recommendation Systems: A Comparative Study of Content-Based Filtering and K-Means Clustering Approaches
- Music genre recognition using convolutional neural networks
- Music Genre Classification with ResNet and Bi-GRU Using Visual Spectrograms
- Convolutional Recurrent Neural Networks for Music Classification

Tools and Frameworks

- **Python**: Core language for orchestrating the entire data pipeline.
- **TensorFlow/Keras**: Framework for building, training, and optimizing the CNN model.
- **Librosa**: Specialized library for audio analysis and mel-spectrogram generation.
- **NumPy and Pandas**: Essential for numerical computations, data manipulation, and metadata handling.
- **Matplotlib and Seaborn**: Used for plotting graphs, confusion matrices, and results visualization.
- **Scikit-learn**: Provided evaluation metrics and additional model utilities.
- **Flask**: Enabled the creation of a simple web interface for delivering recommendations to users.
- **Mutagen**: Assisted in extracting and modifying audio metadata for file organization.
- **OS and Shell Utilities**: Simplified file operations, dataset management, and environment setup.

User Documentation: Setup and Usage Guide

I. Installing Dependencies

pip install numpy pandas librosa matplotlib seaborn scikit-learn tensorflow keras mutagen flask

II. Data Preparation

1. **fma_small Data:** Download from https://github.com/mdeff/fma or the official website and place it in Dataset/fma_small/.
2. **fma_metadata:** Download the metadata (tracks.csv, genres.csv, etc.) and place them in Dataset/fma_metadata/.
3. **DLMusicTest_30:** Prepare a small set of MP3 files you want to test recommendations on and place them in Dataset/DLMusicTest_30/.

III. Splitting and Preparing the Dataset

Run datasplit.py to separate the dataset into training/validation/testing and recommendation subsets. This script also ensures equal genre distribution and moves 20% of songs to Dataset/RecommendMusic_30 for recommendation purposes:

- run python datasplit.py

IV. Renaming MP3 Files in Recommendation Folder (Optional)

If you want to rename the MP3 files in the recommendation folder to human-readable names, run python rename.py

V. Training the Model

To train the CNN model that classifies genres from spectrograms:

run python train.py

What happens when you run train.py:

- **data_importer.py** and **spectogram_slicer.py**:
  The code in train.py automatically calls functions that generate mel-spectrograms from mp3 files (using create_spectrogram()), and then slice them into 128x128 patches (using slice_spect()).
- **data_loads.py**:
  The training script loads the processed spectrogram images and converts them into numpy arrays suitable for training.
- **Model Training**:
  The script trains the CNN, stores the best model in Saved_Model/ and logs the training history.
- After training completes, you will have a saved CNN model in Saved_Model/CNN_Model.keras and a training history CSV file and graphs for reference.

Precomputing Feature Vectors

Once the model is trained, run precompute_features.py to generate latent feature embeddings.

What this does:

- Loads the trained CNN model.
- Uses the model's second-to-last layer to extract feature vectors for each song in the test set (DLMusicTest_30).
- Averages the slice-level features to create one feature vector per track.
- Saves these vectors and corresponding labels in Precomputed_Features/feature_vectors.npy and Precomputed_Features/labels.npy.

Generating Recommendations

With feature vectors precomputed, run recommender.py to interact with the recommendation system

You'll be prompted to enter a seed song name (one present in your DLMusicTest_30 folder). The script then:

- Loads precomputed features.
- Calculates cosine similarities between the seed song and all others.

- Outputs top recommendations and similarity scores.